

Curso de desenvolvemento de aplicacións con tecnoloxías web

Módulo formativo 3: Implantación de aplicacións web en contornos de internet, intranet e extranet

Tema 6: Control de versións

Definición

Características xerais

Tipos de control de versións

Mecanismos de control de versións

Boas prácticas en control de versións

Ferramentas de uso común

Integración con ferramentas de uso común

Definición

O control de versións é unha técnica que permite rexistrar, supervisar e xestionar os cambios realizados nun proxecto, normalmente no código fonte dunha aplicación.

Permite volver a estados anteriores, comparar versións, colaborar entre equipos e documentar a evolución do proxecto.

Características xerais

- Seguimento de cambios co paso do tempo.
- Posibilidade de colaboración entre varios desenvolvedores.
- Rexistro de cambios con mensaxes descritivas.
- Capacidade de reverter erros.
- Integración con ferramentas de integración continua (CI/CD).

Tipos de control de versións

Centralizados

- Exemplo: SVN (Subversion).
- Existe un servidor central que contén o repositorio principal.
- Os usuarios descargan e envían cambios directamente ao servidor.

Pros:

- Sencillez.
- Control centralizado.

Contras:

- Dependencia do servidor central.
- Menor flexibilidade para traballo desconectado.

Distribuídos

- Exemplo: Git.
- Cada usuario ten unha copia completa do repositorio (historial + código).
- Permite traballar offline e sincronizar os cambios máis tarde.

Pros:

- Traballo sen conexión.
- Ramificacións e fusións máis potentes.

Mecanismos de control de versións

Repositorios

- Espazos onde se gardan os arquivos versionados.
- Poden ser locais ou remotos (GitHub, GitLab, Bitbucket...).

Commit (publicación de cambios)

- Operación atómica que garda un conxunto de cambios.
- Require unha mensaxe descritiva.

bash:

```
git add .  
git commit -m "Engadir validación ao formulario de contacto"
```

Checkout (desprotección ou obtención de código)

- Descarga ou cambio a unha rama específica.

bash:

```
git checkout rama-de-novidades
```

Branching (ramificacións)

- Permite desenvolver funcionalidades illadas sen afectar á rama principal (main ou master).

bash:

```
git branch nova-funcionalidade  
git checkout nova-funcionalidade
```

Merging (fusións)

- Unifica cambios dunha rama noutra.

bash:

```
git checkout main  
git merge nova-funcionalidade
```

Tagging (etiquetado)

- Marca unha versión estable (por exemplo, para lanzamentos).

bash:

```
git tag -a v1.0 -m "Primeira versión estable"
```

Baseline (liña de base)

- Punto de partida ben definido desde o cal se mide a evolución dun proxecto.
- Exemplo práctico en Git:
 1. O equipo remata a primeira versión funcional dunha web.
 2. Fan un commit con todos os cambios e crean unha etiqueta (tag) chamada v1.0 → esta é a baseline.
bash:
`git tag -a v1.0 -m "Primeira versión estable"`
`git push origin v1.0`
 3. Despois poden comparar calquera rama ou commit con esa baseline para ver o que cambiou:
bash:
`git diff v1.0..main`

Pull e Update

- `git pull`: actualiza a rama local coa remota.
- `git fetch`: trae cambios sen aplicar.
- `git push`: envía os cambios ao servidor remoto.

Congelacións

- Períodos nos que non se permiten cambios (ex. antes dun lanzamento).

Conflitos

- Producen cando dous usuarios modifican a mesma liña de código.
- Git require que o usuario resolva manualmente.

Boas prácticas en control de versións

- Facer commits pequenos e frecuentes.
- Usar nomes de ramas claros.
- Engadir mensaxes de commit descritivas.
- Evitar subir arquivos innecesarios (usar .gitignore).
- Fusionar cambios regularmente.
- Crear etiquetas para versións estables.

Ferramentas de uso común

Ferramenta	Tipo	Interface gráfica	Almacenamento na nube
Git	Distribuído	CLI / GUI (VS Code, GitKraken)	GitHub, GitLab, Bitbucket
SVN	Centralizado	TortoiseSVN	Non nativo
Mercurial	Distribuído	CLI / GUI	Bitbucket (antes)

Hoxe en día Git é o estándar industrial.

Integración con herramientas de uso común

- Visual Studio Code: integración directa con Git.
- GitHub Desktop: aplicación de escritorio para principiantes.
- GitLab CI/CD: permite automatizar despliegues.
- Extensiones para navegadores e IDEs: permiten ver cambios e colaborar directamente.



Exemplo práctico básico con Git

```
# Inicializar repositorio  
git init
```

```
# Engadir arquivos  
git add .
```

```
# Crear commit  
git commit -m "Primeiro commit"
```

```
# Engadir repositorio remoto (por exemplo GitHub)  
git remote add origin https://github.com/usuario/proxecto.git
```

```
# Subir cambios  
git push -u origin main
```

Exercicio 1: Inicializar e subir un proxecto a un repositorio remoto

Obxectivo:

Practicar os comandos básicos de Git e comprender o fluxo de traballo nun proxecto individual.

Enunciado:

1. Crea unha nova carpeta chamada probas-git.
2. Engade un ficheiro index.html cun contido básico (ex. título e parágrafo).
3. Inicializa un repositorio Git local.
4. Fai un primeiro commit.
5. Crea un repositorio remoto en GitHub, GitLab ou Plesk (segundo o caso).
6. Engade o remoto e sobe os cambios.

Exercicio 2: Ramificación e fusión

Obxectivo:

Aprender a crear ramas, traballar nelas e unilas á rama principal.

Enunciado:

1. No repositorio anterior, crea unha rama nova chamada estilo.
2. Nesa rama, crea un ficheiro estilos.css e enlázao no index.html.
3. Fai un commit cos cambios.
4. Volve á rama main e fusionaa coa rama estilo.

Exercicio 3: Resolución de conflitos

Obxectivo:

Simular un conflito de fusión e resolvelo manualmente.

Enunciado:

1. En main, edita index.html e cambia o título da páxina.
2. Crea unha nova rama cambio-paragrafo e edita o mesmo título con outro texto distinto.
3. Fai commit e intenta facer merge a main.

Debería aparecer un conflito que o alumnado debe resolver editando o ficheiro e facendo un novo commit.

Exercicio 4: Uso de .gitignore e etiquetas

Obxectivo:

Aprender a excluír ficheiros innecesarios e a etiquetar versións.

Enunciado:

1. Crea un ficheiro notas.txt e engádeo ao .gitignore.
2. Crea unha carpeta node_modules/ (ou similar) e exclúe tamén.
3. Fai un commit.
4. Crea unha etiqueta v1.0 no punto actual.

Exercicio 5 (opcional): Colaboración por parellas

Obxectivo:

Practicar o traballo colaborativo con ramas e pull requests.

Enunciado:

1. Un alumno crea un repositorio remoto e comparte acceso co compañeiro.
2. Cada alumno traballa nunha rama distinta.
3. Simulan un fluxo de colaboración: edición → commit → push → merge.